UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época Recurso • 29 de janeiro de 2023 • 16:00 – 18:00

- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas lápis/caneta, identificação, este enunciado, e folha de teste.

Grupo I: Perguntas de escolha múltipla

1. (0.5) Considere o seguinte código

```
public static Integer xpto(Integer i)
{
    Integer zero = new Integer(0);
    Integer ten = new Integer(10);
    return new Integer(i.intValue()+ten.intValue());
}
public static void main(String[] args)
{
    Integer i1,i2,i3,i4,i5;

    i1 = new Integer(12);
    i1 = xpto(i1);
    i2 = xpto(i1);
    i3 = xpto(i2);
    i2 = null;
}
```

No fim da execução da última linha do método main, assumindo que o coletor de lixo é invocado, quantos objetos foram criados no *heap*, e quantos foram marcados para remoção?

a) 10 criados, 7 marcados para remoção.
b) 9 criados, 8 marcados para remoção.
c) 7 criados, 7 marcados para remoção
d) 10 criados, 8 marcados para remoção

2. (0.5) Considere as seguintes afirmações:
1. O algoritmo *insertionSort* tem complexidade temporal $O(n^2)$.
2. O algoritmo insertionSort tem complexidade temporal $O(n \log_2 n)$.
3. Se o array estiver ordenado de forma crescente, o algoritmo insertion sort é linear em n.
4. Se o array estiver ordenado de forma decrescente, o algoritmo insertion sort é linear em n.

Quais das afirmações acima estão corretas?

a) nenhuma.
b) 1 e 3
c) 1 e 4
d) 2 e 3
e) 2 e 4

3. (0.5) Considere o algoritmo de ordenação Shellsort usando a sequência de Knuth aplicado ao seguinte array [4, 2, 5, 0, 1, 7, 3, 8]. Indique qual das seguintes sequências corresponde a uma sequência válida e pela ordem correta do estado do array após operações de h-sorting. Assuma que h inicial = 4.

a)
[0, 1, 2, 3, 4, 5, 7, 8]
[0, 1, 2, 3, 4, 5, 7, 8]

b)
[1, 2, 3, 0, 4, 7, 5, 8]
[0, 2, 3, 1, 4, 7, 5, 8]

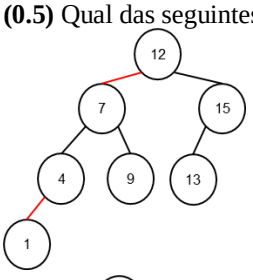
c)
[1, 2, 3, 0, 4, 7, 5, 8]
[0, 1, 2, 3, 4, 5, 7, 8]

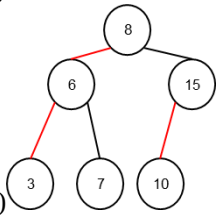
d)
[1, 0, 3, 2, 4, 7, 5, 8]
[0, 1, 2, 3, 4, 5, 7, 8]

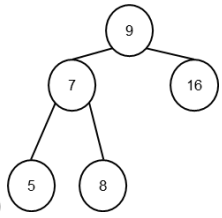
4. (0.5) Qual dos seguintes arrays é um max-heap válido? Assuma que os elementos começam na posição 1 do array.

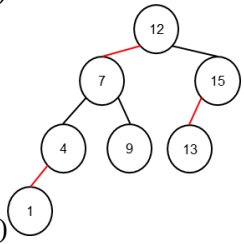
a) [null, 9, 3, 8, 1, 2, 8, 7]
b) [null, 9, 7, 6, 2, 3, 8, 5]
c) [null, 1, 2, 3, 4, 5, 6, 7]
d) [null, 7, 9, 8, 1, 2, 2, 3]

5. (0.5) Qual das seguintes árvores é uma árvore red-black válida?

a)


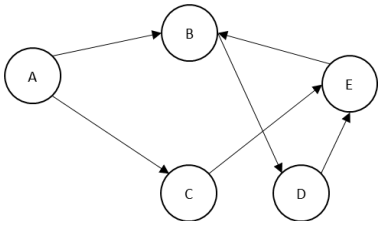
b)


c)


d)


e) Nenhuma das árvores apresentadas é uma árvore red-black válida

6. (0.5) Considere o grafo dirigido representado abaixo. Indique o número de componentes fortemente ligados do grafo.



Selecione uma opção de resposta:

a) 1
b) 2
c) 3
d) 4
e) 5

Grupo II: Perguntas de desenvolvimento

7. (3.0) Exercícios sobre a linguagem de programação Java e complexidade temporal.

a) (1.0) Escreva um método estático chamado contem, que recebe um inteiro e, e um array de inteiros a, e devolve o valor lógico verdadeiro se o elemento e estiver dentro do array, e o valor lógico falso se e não estiver.dentro do array.

b) (1.0) Usando o método implementado na alínea anterior, implemente um método estático, designado de *intersecao* que recebe dois *arrays* de inteiros *a1* e *a2*, e retorna um novo array de inteiros cujos elementos correspondem à interseção dos elementos de a1 e a2 (por outra palavras retorna os elementos que estão simultaneamente em a1 e a2). Por exemplo intersecao([1,2,3],[4,5,6]) devolverá o array vazio [], e intersecao([1,2,3,4],[2,5,4]) retornará [2,4].

c) (1.0) Indique qual a complexidade temporal assimptótica do seu método contem em função do tamanho do array n. Indique qual a complexidade temporal assimptótica do seu método intersecao em função do tamanho dos arrays *a1* e *a2* (pode assumir que *a1* e *a2* têm o mesmo tamanho n).

8. (2.0) Considere uma pilha (*stack*) e uma **fila prioritária** (*priority queue*) organizada pelo máximo elemento, inicialmente vazias, sobre as quais é executada a seguinte sequência de instruções:

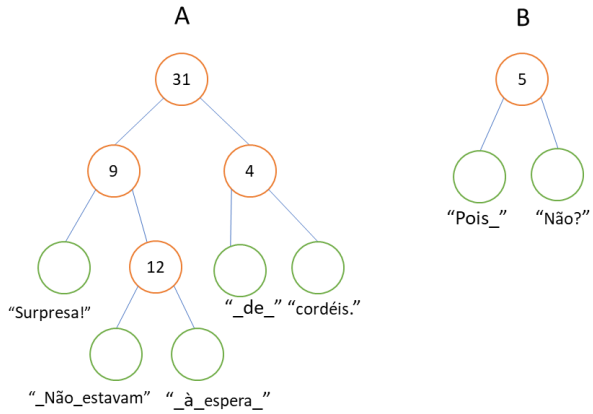
```
StackList<Integer> s = new StackList<Integer>();
MaxPriorityQueue<Integer> q = new MaxPriorityQueue<Integer>();

s.push(7);
s.push(4);
q.enqueue(2);
q.enqueue(6);

for(int i = 0; i<3; i++)
{
    s.push(q.dequeue());
    q.enqueue(i);
    s.push(q.dequeue());
    q.enqueue(s.pop());
    q.enqueue(s.pop());
}
while(!s.isEmpty()) System.out.println(s.pop() + "/");
```

Represente a fila prioritária q e a pilha s, com os elementos introduzidos e removidos ao longo da execução do código e indique a sequência impressa no ecrã quando o código é executado.

9. (2.5) Considere a estrutura de dados Cordel, implementada no Projeto 3. Dados os seguintes cordéis A e B:



a) (1.0) Indique quais os nós de concatenação e os nós folha visitados quando é efetuada a seguinte operação sobre o Cordel a: *a.charAt(13)*. Indique também o caracter retornado pelo método.

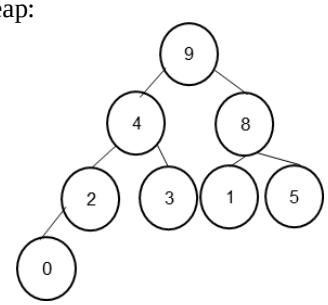
b) (1.0) Desenhe o cordel que se obtém ao fazer as seguintes instruções: *a.merge(b.prepend("abc"))*.

10. (2.0) Considere o método *partition* usado no algoritmo de ordenação ***quicksort***, com a assinatura *int[] partition(Comparable[] a, int low, int high)*. Suponha que o método *partition é* invocado com os seguintes argumentos:

a=[5, 9, 1, 7, 4, 6, 3, 2, 8,0], low=0,high=9

Indique os vários passos do método *partition*, indicando o conteúdo do *array* após cada troca de elementos, assumindo que o pivô é escolhido como sendo o elemento da posição *low*. Indique também qual a posição do pivô, retornado pelo método.

11. (2.5) Considere o seguinte max-heap:



a) (1.5) Desenhe a representação em árvore do *max-heap* obtido quando se insere os elementos 6 e 11 no heap. Desenhe os principais passos intermédios.

b) (1.0) Desenhe a representação em árvore do *max-heap* obtido quando se remove o maior elemento do ***heap* original**. Desenhe os principais passos intermédios.

12. (1.0) Indique, justificando, quais as vantagens de um amontoado (*heap*) face a uma árvore binária de pesquisa.

13. (1.0) Considere que necessita de implementar uma procura em profundidade primeiro, sobre um grafo, sem usar recursividade. Isto pode ser feito usando uma coleção para guardar os nós ainda não visitados, e quando necessitamos de escolher o próximo nó a visitar, escolhemos o último nó que foi adicionado à coleção, removendo-o. Que estrutura de dados usaria para implementar esta coleção? Justifique.

14. (2.5) Considere uma tabela de dispersão de dimensão m = 9, com resolução de colisões por dupla dispersão, e funções de dispersão h1(k) = k mod m e h2(k) = 7 – k mod 7
a) (1.5) Sabendo que que inicialmente a tabela se encontra preenchida com as chaves indicadas, indique quais os elementos presentes em cada posição do array de chaves e de valores, apos a inserção da sequência de pares <chave , valor> abaixo. **Indique o valor de h1 e h2 calculado para cada chave**. Indique uma posição vazia com ▬.

<3 , “a”>, <6 , “b”>, <12 , “c”>, < 15, “d”>, <21 , “e”> ,

i	0	1	2	3	4	5	6	7	8
chaves	18			21					
valores	“k”			a					

b) (1.0) O valor escolhido como tamanho da tabela é adequado? Justifique.

15. (2.5) Considere a execução do algoritmo de Prim sobre o grafo pesado não dirigido da figura abaixo. Qual a sequência pela qual o algoritmo de Prim visita as arestas de corte, assumindo que o algoritmo inicia a procura a partir do **vértice A**? (Em caso de empate, utilize a ordem alfabética dos vértices para desempatar). Desenhe a árvore de cobertura mínima obtida pelo algoritmo.

